Name: Soham Chavan

PRN: 124B1F022

Class: IT A

Assignment No. 1

Aim: To design and implement Merge Sort to arrange customer orders based on their timestamps efficiently, and to analyze its time complexity in comparison with other traditional sorting techniques.

Code:

```
import java.util.Random;

class Order {
    int orderId;
    long timestamp;

    Order(int orderId, long timestamp) {
        this.orderId = orderId;
        this.timestamp = timestamp;
    }

    public String toString() {
        return "OrderID:" + orderId + " Timestamp:" + timestamp;
    }
}

public class MergeSortOrders {

    public static void mergeSort(Order[] orders, int left, int right) {
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(orders, left, mid);
            mergeSort(orders, mid + 1, right);
            merge(orders, left, mid, right);
        }
    }

    public static void merge(Order[] orders, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        Order[] leftArr = new Order[n1];
        Order[] rightArr = new Order[n2];

        for (int i = 0; i < n1; i++) leftArr[i] = orders[left + i];
        for (int j = 0; j < n2; j++) rightArr[j] = orders[mid + 1 + j];

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (leftArr[i].timestamp <= rightArr[j].timestamp) {
                orders[k] = leftArr[i];
```

```
            i++;
        } else {
            orders[k] = rightArr[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        orders[k] = leftArr[i];
        i++;
        k++;
    }

    while (j < n2) {
        orders[k] = rightArr[j];
        j++;
        k++;
    }
}

public static void main(String[] args) {
    Random rand = new Random(42);

    int smallSize = 10;
    Order[] smallOrders = new Order[smallSize];
    for (int i = 0; i < smallSize; i++) {
        smallOrders[i] = new Order(i + 1, 1000 + rand.nextInt(9000));
    }

    System.out.println("Unsorted Orders:");
    for (Order o : smallOrders) System.out.println(o);

    mergeSort(smallOrders, 0, smallOrders.length - 1);

    System.out.println("\nSorted Orders (by Timestamp):");
    for (Order o : smallOrders) System.out.println(o);

    int largeSize = 1000000;
    Order[] largeOrders = new Order[largeSize];
    for (int i = 0; i < largeSize; i++) {
        largeOrders[i] = new Order(i + 1, rand.nextLong(1000000000L));
    }

    long startTime = System.nanoTime();
    mergeSort(largeOrders, 0, largeOrders.length - 1);
    long endTime = System.nanoTime();

    System.out.println("\nSorted " + largeSize + " orders successfully.");
    System.out.println("Time taken: " + (endTime - startTime) / 1000000.0 + " ms");

    boolean isSorted = true;
    for (int i = 0; i < largeOrders.length - 1; i++) {
        if (largeOrders[i].timestamp > largeOrders[i + 1].timestamp) {
            isSorted = false;
            break;
        }
    }
    System.out.println("Array correctly sorted: " + isSorted);
}
```

```
}
```

Output:

```
Unsorted Orders:
OrderID:1 Timestamp:5130
OrderID:2 Timestamp:6763
OrderID:3 Timestamp:5248
OrderID:4 Timestamp:7884
OrderID:5 Timestamp:3970
OrderID:6 Timestamp:5525
OrderID:7 Timestamp:5505
OrderID:8 Timestamp:3918
OrderID:9 Timestamp:2519
OrderID:10 Timestamp:9093

Sorted Orders (by Timestamp):
OrderID:9 Timestamp:2519
OrderID:8 Timestamp:3918
OrderID:5 Timestamp:3970
OrderID:1 Timestamp:5130
OrderID:3 Timestamp:5248
OrderID:7 Timestamp:5505
OrderID:6 Timestamp:5525
OrderID:2 Timestamp:6763
OrderID:4 Timestamp:7884
OrderID:10 Timestamp:9093

Sorted 1000000 orders successfully.
Time taken: 264.332448 ms
Array correctly sorted: true
```

Conclusion:

Merge Sort was successfully implemented to sort customer orders by timestamp. It follows the divide-and-conquer approach, splitting the array into halves recursively and merging them in sorted order. Its time complexity is O(n log n) in the best, average and worst case, which makes it much more reliable for large datasets (up to 1 million orders) compared to O(n^2) algorithms like Bubble Sort or Insertion Sort. Its stability and predictable performance make it well suited for sorting order records by timestamp.